

# Backend Julia

Guia de estudo passo a passo — do zero ao pedido, explicado para leigos

Este documento explica, em ordem, tudo o que foi construído: uma **API de backend para uma sorveteria** (cadastro de clientes, login, recuperação de senha, produtos, sabores, carrinho e pedidos), usando **DDD**, **SOLID** e **Clean Architecture**. Cada etapa tem o "o quê", o "por quê" e uma analogia do mundo real.

---

## Sumário

---

Parte 1 — Conceitos que você precisa antes de começar

Parte 2 — A ideia da arquitetura em camadas

Parte 3 — As etapas da construção

Etapa 0 — Entender o projeto que já existia

Etapa 1 — Modelar o banco de dados

Etapa 2 — Gerar a migration e o cliente Prisma

Etapa 3 — O "kit compartilhado" (Either, Erros, Money)

Etapa 4 — Domínio: Cliente

Etapa 5 — Domínio: Código de recuperação de senha

Etapa 6 — Domínio: Produto e Sabor

Etapa 7 — Domínio: Carrinho

Etapa 8 — Domínio: Pedido

Etapa 9 — Contratos de repositório

Etapa 10 — Ports (contratos de infraestrutura)

Etapa 11 — Casos de uso

Etapa 12 — Providers (as ferramentas reais)

Etapa 13 — Repositórios com Prisma

Etapa 14 — A camada HTTP (Fastify)

Etapa 15 — O container (composition root)

Etapa 16 — Servidor, variáveis de ambiente e seed

Etapa 17 — Ajuste do TypeScript

Etapa 18 — Testes

Parte 4 — Fluxos completos narrados

Parte 5 — Mapa de endpoints

Parte 6 — Como rodar

Parte 7 — Glossário

Parte 8 — Exercícios para fixar

# Parte 1 — Conceitos que você precisa antes de começar

## O que é um "backend"?

Quando você usa um aplicativo (um site de loja, por exemplo), existem **duas metades**:

- **Frontend**: a tela que você vê e toca (botões, listas, formulários). Roda no celular ou no navegador.
- **Backend**: o "cérebro" que fica num servidor. Ele guarda os dados, aplica as regras ("o carrinho não pode ter quantidade negativa"), confere senha, calcula o total do pedido, etc.

Este projeto é **só o backend**. Ele não tem telas. Ele oferece uma **API**.

 **Analogia** — Pense num restaurante. O **frontend** é o salão e o cardápio. O **backend** é a cozinha. Você não entra na cozinha; você faz um *pedido* pelo garçom e recebe um *prato* de volta.

## O que é uma API, uma rota e um método HTTP?

**API** é o conjunto de "pedidos" que o backend aceita. Cada pedido vai para uma **rota** (um endereço, tipo `/products` ) usando um **método HTTP** que indica a intenção:

| Método | Intenção                        | Exemplo neste projeto                                 |
|--------|---------------------------------|-------------------------------------------------------|
| GET    | Ler / consultar                 | GET <code>/products</code> — listar produtos          |
| POST   | Criar algo novo                 | POST <code>/customers</code> — cadastrar cliente      |
| PUT    | Substituir / editar por inteiro | PUT <code>/admin/products/:id</code>                  |
| PATCH  | Editar um pedacinho             | PATCH <code>/cart/items/:id</code> — mudar quantidade |
| DELETE | Remover                         | DELETE <code>/cart</code> — esvaziar carrinho         |

Os dados vão e voltam em **JSON**, um formato de texto simples com pares "chave: valor". Exemplo do corpo de um cadastro:

```
{ "name": "Maria", "email": "maria@exemplo.com", "password": "senha1234" }
```

A resposta traz um **código de status**: `200/201` = deu certo, `400` = você mandou dado inválido, `401` = não está logado, `403` = logado mas sem permissão, `404` = não existe, `409` = conflito (ex.: e-mail já cadastrado), `500` = erro nosso.

## O que é um banco de dados e um ORM?

**Banco de dados** é onde os dados ficam guardados de forma permanente, em **tabelas** (parecidas com planilhas: colunas e linhas). Aqui usamos o **SQLite**, um banco que é um único arquivo ( `dev.db` ) — ótimo para estudar.

Falar com o banco "na mão" exige escrever **SQL**. Um **ORM** ("Object-Relational Mapper") faz essa ponte para você: você escreve código em TypeScript e ele gera o SQL. Aqui o ORM é o **Prisma**.

🗨️ **Analogia** — O banco é um **arquivo de aço** com gavetas e pastas. O ORM (Prisma) é um **assistente** que busca e guarda pastas para você quando você pede em português, sem você precisar saber em qual gaveta está.

## O que é autenticação por "token"?

Depois que o cliente faz login, o backend devolve um **token**: um texto embaralhado que funciona como uma **pulseira de evento**. Em cada pedido seguinte, o app manda essa pulseira no cabeçalho `Authorization: Bearer <token>`. O backend confere se a pulseira é autêntica (não foi falsificada) e não expirou. Se estiver ok, ele sabe *quem* é você sem pedir a senha de novo.

## DDD, SOLID e Clean Architecture — sem jargão

São três formas de **organizar o código** para ele não virar um romance.

- **DDD (Domain-Driven Design)**: coloque as **regras do negócio** no centro e dê a elas nomes do mundo real ("Cliente", "Carrinho", "Pedido", "Sabor"). O código deve parecer com a forma como as pessoas do negócio falam.
- **SOLID**: cinco princípios para módulos pequenos e trocáveis. O mais importante aqui: **dependa de contratos, não de detalhes**. O código que decide "como cadastrar um cliente" não sabe se a senha é criptografada com a ferramenta X ou Y — ele só conhece um contrato "algo que sabe criptografar".
- **Clean Architecture**: desenhe o sistema em **camadas concêntricas**. O miolo (regras) não conhece o mundo externo (banco, internet). As setas de dependência apontam sempre *para dentro*.

❓ **Por que fizemos assim** — Se amanhã a sorveteria trocar SQLite por PostgreSQL, ou o e-mail por outro serviço, ou o formato do token — **as regras de negócio não mudam nem uma linha**. Só troca a "peça" da borda. É esse o ganho de separar em camadas.

## Parte 2 — A ideia da arquitetura em camadas

Imagine a empresa "Sorveteria Julia" com quatro setores. Cada arquivo do projeto mora em um deles:

| Camada (pasta)                                           | Analogia                                                       | Contém                                                                                                                 | Pode depender de...                                      |
|----------------------------------------------------------|----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| <b>domínio</b><br><code>src/domain</code>                | O "livro de regras" da empresa                                 | Entidades (Cliente, Produto, Carrinho, Pedido), objetos de valor (E-mail, Dinheiro) e <i>interfaces</i> de repositório | <b>Nada.</b> Não sabe o que é Fastify nem Prisma.        |
| <b>aplicação</b><br><code>src/application</code>         | Os "procedimentos operacionais" (passo a passo de cada tarefa) | Casos de uso ("Cadastrar cliente", "Criar pedido") e <i>ports</i> (contratos das ferramentas)                          | Só do domínio e dos próprios contratos.                  |
| <b>infraestrutura</b><br><code>src/infra</code>          | As "ferramentas e fornecedores" reais                          | Prisma (banco), gerador de token, envio de e-mail, e o servidor HTTP (Fastify) com as rotas                            | De tudo — ela é a borda. <i>Implementa</i> os contratos. |
| <b>composition root</b><br><code>src/container.ts</code> | O "RH" que contrata e conecta todo mundo no primeiro dia       | Cria as peças concretas e injeta umas nas outras                                                                       | De todas.                                                |

 **Dica** — Regra de ouro para checar se está certo: **um arquivo em `domain/` nunca deve ter `import de fastify`, `@prisma/client` ou de qualquer coisa em `infra/`**. Se tiver, a dependência apontou para fora — errado.

## Estrutura final de pastas

```
src/  
  shared/           Either, Erros, Money (usados por todas as camadas)  
  domain/  
    customer/       Customer, Email, política de senha, contrato de repositório  
    auth/           PasswordResetCode + contrato  
    catalog/        Product, Flavor + contratos  
    cart/           Cart, CartItem + contrato  
    order/          Order, OrderItem + contrato  
  application/  
    ports/          HashProvider, TokenProvider, MailProvider, geradores  
    shared/         paginação  
    customer/       register-customer, authenticate-customer  
    auth/           request-password-reset, reset-password  
    product/        create / update / list / get  
    flavor/         create / update / list  
    cart/           get / add-item / update-item / remove-item / clear + cart-view  
    order/          create / list / get  
  infra/  
    database/prisma/ client + 6 repositórios  
    providers/      scrypt (hash), HMAC (token), console (e-mail), uuid/código  
    http/           app, auth (guards), http-helpers, routes/*  
  config/env.ts     leitura das variáveis de ambiente  
  container.ts      liga tudo  
  server.ts         liga o servidor  
prisma/  
  schema.prisma     desenho do banco  
  seed.ts           cria o admin inicial
```

## Parte 3 — As etapas da construção

### 0 Entender o projeto que já existia

**O quê:** antes de escrever qualquer coisa, olhamos o que já estava na pasta.

**O que encontramos:** um esqueleto — Fastify + Prisma + SQLite já instalados, um `schema.prisma` com só a tabela `Customer` (sem senha!), um arquivo de rotas de pedido com respostas "de mentira" ( `"pedido criado"` ) e um caso de uso vazio.

**Por quê essa etapa:** reaproveitar as escolhas do projeto (linguagem, framework, banco, forma de importar) em vez de impor outras. O código novo tem que "parecer" com o que já existe.

### 1 Modelar o banco de dados

**O quê:** reescrevemos `prisma/schema.prisma` descrevendo todas as tabelas ("models") e como elas se ligam.

**Tabelas criadas e para que servem:**

| Tabela                         | Guarda                                                        | Campos-chave                                                                                                              |
|--------------------------------|---------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>Customer</code>          | O cliente (e também o admin)                                  | <code>email</code> único, <code>passwordHash</code> , <code>role</code> = <code>"CUSTOMER"</code> ou <code>"ADMIN"</code> |
| <code>PasswordResetCode</code> | Código de recuperação de senha, já com hash                   | <code>codeHash</code> , <code>expiresAt</code> , <code>consumedAt</code>                                                  |
| <code>Product</code>           | Um produto do catálogo                                        | <code>priceCents</code> (preço em centavos), <code>active</code>                                                          |
| <code>Flavor</code>            | Um sabor                                                      | <code>name</code> único, <code>active</code>                                                                              |
| <code>ProductFlavor</code>     | Ligação "este produto aceita este sabor" (muitos-para-muitos) | <code>par</code> ( <code>productId</code> , <code>flavorId</code> )                                                       |
| <code>Cart</code>              | O carrinho aberto do cliente (um por cliente)                 | <code>customerId</code> único                                                                                             |
| <code>CartItem</code>          | Uma linha do carrinho                                         | <code>quantity</code> , <code>unitPriceCents</code>                                                                       |
| <code>Order</code>             | Um pedido fechado                                             | <code>status</code> , <code>totalCents</code>                                                                             |
| <code>OrderItem</code>         | Uma linha do pedido, com nome e preço "congelados"            | <code>productName</code> , <code>flavorName</code> , <code>unitPriceCents</code>                                          |

**? Por que fizemos assim — Preço em centavos (número inteiro).** Computador erra conta com "vírgula" (0,1 + 0,2 não dá exatamente 0,3). Guardando 2500 em vez de 25.00, somas e multiplicações ficam exatas. Convertendo para reais só na hora de exibir.

? **Por que fizemos assim** — `Admin` é um `Customer` com `role = "ADMIN"`. Não criamos uma tabela separada de usuários. É mais simples e o enunciado só fala em "cliente" e "administrador". O SQLite não tem tipo "enum", então `role` e `status` são texto, validados no código.

? **Por que fizemos assim** — `OrderItem` copia o nome e o preço do produto. Se amanhã o produto "Sorvete 1L" mudar de preço ou for apagado, o pedido antigo tem que continuar mostrando o que foi realmente comprado e por quanto. Isso se chama *snapshot* (foto do momento).

## 2 Gerar a migration e o cliente Prisma

O quê: rodamos dois comandos.

```
npx prisma migrate dev --name full_domain # cria as tabelas de verdade no dev.db
npx prisma generate                      # gera código TypeScript com os tipos das tabelas
```

- **Migration** = um arquivo `.sql` versionado que descreve a mudança no banco. Fica em `prisma/migrations/`. Qualquer pessoa do time roda a mesma migration e fica com o banco idêntico.
- **Generate** = o Prisma lê o schema e escreve, em `generated/client`, um objeto `prisma` com métodos como `prisma.customer.findUnique(...)` já sabendo os campos.

## 3 O "kit compartilhado": Either, Erros e Money

O quê: três arquivos em `src/shared/` usados por todo o resto.

### 1) `errors.ts` — erros com "etiqueta"

Uma família de classes de erro: `ValidationError` (400), `UnauthorizedError` (401), `ForbiddenError` (403), `NotFoundError` (404), `ConflictError` (409). Cada uma carrega um `status` (número HTTP) e um `code` (texto estável, ex.: `"CONFLICT"`).

? **Por que fizemos assim** — Assim a camada HTTP não precisa de um `if/else` gigante para decidir o status. Ela pega o erro, lê `erro.status` e `erro.code` e responde. O caso de uso decide o "significado", o HTTP só traduz.

### 2) `either.ts` — devolver "deu certo" ou "deu erro" sem lançar exceção

`Either` é uma caixinha que ou tem `{ ok: false, value: erro }` ou `{ ok: true, value: resultado }`. Funções auxiliares: `left(erro)` e `right(valor)`.

🗨️ **Analogia** — É como um **resultado de exame**: vem *sempre* um envelope. Dentro, ou está "alterado, procure o médico" ou "tudo normal, aqui estão os números". Você é obrigado a abrir e checar qual é antes de seguir.

? **Por que fizemos assim** — Com `throw`, um erro "voa" e pode ser esquecido. Com `Either`, o TypeScript te obriga a checar `if (resultado.ok)` antes de usar o valor. Fluxos previsíveis de negócio ("e-mail já existe") não são "exceções" — são respostas normais.

### 3) `money.ts` — o objeto de valor Dinheiro

Uma classe `Money` que guarda `cents` (inteiro) e só deixa criar valor válido ( `Money.fromCents` recusa negativo ou quebrado). Tem `add()`, `multiply(qtd)` e `reais`.

💡 **Dica — Objeto de valor** = algo definido pelo seu conteúdo, não por um "id". Dois `Money` de 2500 centavos são "o mesmo dinheiro". Ele é *imutável*: somar gera um novo `Money`, não altera o antigo.

## 4 Domínio: Cliente

Quatro arquivos em `src/domain/customer/`.

- `email.ts` — objeto de valor **Email**. `Email.create(" MARIA@X.COM ")` apara espaços, deixa minúsculo, confere o formato e devolve `Either`. Depois disso, é impossível ter um e-mail "meio inválido" circulando no sistema.
- `password-policy.ts` — objeto de valor **RawPassword** (senha em texto puro). Só aceita se tiver 8+ caracteres com letras e números. Vive pouquíssimo tempo: nasce, é conferida, é transformada em hash e some.
- `customer.ts` — a **entidade Customer**. Tem `id`, `name`, `email`, `phone`, `passwordHash`, `role`, `datas`. Construtor é *privado*; cria-se por `Customer.create(...)` (novo cliente) ou `Customer.restore(...)` (reconstruído a partir do banco). Métodos de negócio: `changePassword()`, `updateProfile()`, `isAdmin`, e `toPublic()` que devolve os dados **sem o hash da senha**.
- `customer.repository.ts` — só a **interface CustomerRepository**: "quem quiser guardar clientes precisa saber fazer `findById`, `findByEmail`, `create`, `save`". Não diz *como*.

? **Por que fizemos assim** — Entidade "rica", não "anêmica". Em vez de um saco de dados que qualquer um altera à vontade, a entidade protege suas próprias regras. Você não faz `customer.role = "ADMIN"` solto; a entidade controla o que pode mudar e quando.

## 5 Domínio: Código de recuperação de senha

`src/domain/auth/`: a entidade **PasswordResetCode** e sua interface de repositório.

Regras que a entidade garante sozinha:

- Nasce com `expiresAt` = agora + 15 min (configurável).
- `isExpired()`, `isConsumed()`, `isUsable()` respondem sobre o estado.
- `consume()` marca como usado — o código vale **uma vez só**.
- O que se guarda é o **hash** do código, nunca o número em si.

**? Por que fizemos assim** — Se o banco vazar, ninguém consegue os códigos de recuperação (são só hashes). E como expira e é de uso único, uma pessoa mal-intencionada tem uma janela minúscula mesmo que intercepte o e-mail.

## 6 Domínio: Produto e Sabor

`src/domain/catalog/`.

- `flavor.ts` — entidade **Flavor**: `name`, `active`, `update()`.
- `product.ts` — entidade **Product**: `name`, `description`, `price` (um `Money`), `active`, e `flavorIds` (lista de sabores que ele aceita). Métodos de negócio úteis: `offersFlavor(id)` ("este produto aceita esse sabor?") e `requiresFlavor` ("este produto exige escolher um sabor?").
- `catalog.repository.ts` — interfaces `ProductRepository` e `FlavorRepository`, mais os tipos de **paginação** (`page`, `perPage`, `total`).

## 7 Domínio: Carrinho

`src/domain/cart/cart.ts` define duas classes que trabalham juntas — um **agregado**:

- **CartItem**: uma linha (produto + sabor opcional + quantidade + preço unitário). Sabe calcular seu `subtotal`.
- **Cart**: a "raiz". Todo mundo mexe no carrinho *através* dele, nunca direto nos itens. Ele garante as regras:
  - `addItem(...)`: se já existe uma linha com o *mesmo produto e mesmo sabor*, só aumenta a quantidade; senão cria uma linha nova.
  - `changeItemQuantity(id, q)`: recusa quantidade menor que 1; devolve `NotFoundError` se o item não existe.
  - `removeItem(id)`, `clear()`, `total` (soma dos subtotais como `Money`), `isEmpty`.

**Analogia** — O **agregado** é como uma *ata de reunião*: você não risca uma linha no meio à caneta. Você fala com o secretário (a raiz `Cart`), e ele altera de forma controlada para o documento nunca ficar inconsistente.

## 8 Domínio: Pedido

`src/domain/order/order.ts` : agregado **Order** + **OrderItem**.

- `Order.createFromItems( ... )` recebe a lista de itens "congelados" e devolve `Either` — recusa criar pedido **sem nenhum item**.
- Cada `OrderItem` guarda `productName` e `flavorName` (texto), não só os ids. É a "foto" do momento da compra.
- `status` começa em `"PENDING"`. Métodos `markPaid()` e `cancel()` aplicam as regras de transição (ex.: não dá para cancelar pedido já pago).
- `total` é a soma dos subtotais.

## 9 Contratos de repositório

**O quê:** em cada pasta do domínio há um arquivo `*.repository.ts` que é **só uma interface** (uma lista de métodos, sem corpo).

```
export interface CustomerRepository {
  findById(id: string): Promise<Customer | null>
  findByEmail(email: string): Promise<Customer | null>
  create(customer: Customer): Promise<void>
  save(customer: Customer): Promise<void>
}
```

**? Por que fizemos assim** — Isto é o **"D" de SOLID** (Inversão de Dependência). O domínio *declara o que precisa* ("preciso poder achar cliente por e-mail"). Quem cumpre essa promessa usando Prisma vem depois, na infraestrutura. O domínio nunca "olha para baixo".

## 10 Ports (contratos de infraestrutura)

`src/application/ports/` — mais interfaces, agora para **ferramentas técnicas** que os casos de uso precisam:

| Port                                                  | Promessa                                                                                            |
|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>HashProvider</code>                             | <code>hash(texto)</code> e <code>compare(texto, hash)</code> — embaralhar senhas/códigos e conferir |
| <code>TokenProvider</code>                            | <code>sign(dados)</code> e <code>verify(token)</code> — criar e validar a "pulseira" do login       |
| <code>MailProvider</code>                             | <code>send(mensagem)</code> — enviar e-mail                                                         |
| <code>IdGenerator</code> / <code>CodeGenerator</code> | gerar identificadores únicos e o código numérico de 6 dígitos                                       |

? **Por que fizemos assim — Interface Segregation (o "I" de SOLID):** cada port é minúsculo e faz uma coisa. Um caso de uso que só precisa enviar e-mail recebe só o `MailProvider`, não um "objeto" com 40 métodos.

## 11 Casos de uso

O quê: em `src/application/<assunto>/`, uma classe por tarefa. Todas seguem o mesmo molde:

```
class FazAlgo {
  constructor(private repo: AlgunRepositório, private ferramenta: AlgunPort) {}
  async execute(input): Promise<Either<AppError, Output>> {
    // 1. validar a entrada
    // 2. buscar o que precisa nos repositórios
    // 3. aplicar a regra chamando métodos das entidades
    // 4. salvar
    // 5. devolver right(resultado) ou left(erro)
  }
}
```

A lista completa e o que cada um faz:

### Cliente e login

- **RegisterCustomer** ( `POST /customers` ) — valida e-mail e senha (objetos de valor), confere se o e-mail já existe ( `ConflictError` ), gera o hash da senha, cria a entidade `Customer` e salva.
- **AuthenticateCustomer** ( `POST /sessions` ) — acha o cliente por e-mail, compara a senha com o hash. Se qualquer coisa falhar, devolve **a mesma** mensagem genérica ("E-mail ou senha inválidos") — não conta se foi o e-mail ou a senha que errou. Se der certo, assina um token com `{ sub: id, role }`.

### Recuperação de senha

- **RequestPasswordReset** ( `POST /password/forgot` ) — acha o cliente. **Se não existir, finge que deu certo** (não revela quem tem conta). Se existir: invalida códigos antigos, gera um código de 6 dígitos, guarda só o *hash* dele com validade de 15 min, e manda o código por e-mail.
- **ResetPassword** ( `POST /password/reset` ) — valida a nova senha, pega o último código utilizável do cliente, compara com o que ele enviou. Se ok: troca a senha (novo hash), marca o código como *consumido* e invalida os demais.

### Produtos (área do administrador)

- **CreateProduct** ( `POST /admin/products` ) — valida nome e preço ( `Money.fromCents` ), confere se todos os `flavorIds` informados existem de verdade, cria o produto.
- **UpdateProduct** ( `PUT /admin/products/:id` ) — acha o produto ( `NotFoundError` se não achar), aplica só os campos enviados, revalida sabores se a lista mudou.
- **ListProducts** ( `GET /products` e `GET /admin/products` ) — lista paginada. O público só vê ativos; o admin pode ver inativos também.

- **GetProduct** ( `GET /products/:id` ) — um produto pelo id.

### Sabores (área do administrador)

- **CreateFlavor** ( `POST /admin/flavors` ) — valida nome, recusa nome repetido ( `ConflictError` ).
- **UpdateFlavor** ( `PUT /admin/flavors/:id` ) — edita nome/ativo, checando de novo por nome repetido.
- **ListFlavors** ( `GET /flavors` , `GET /admin/flavors` ) — paginada, mesma ideia de público × admin.

### Carrinho (precisa estar logado)

- **GetCart** — devolve o carrinho; se não existir, cria um vazio.
- **AddCartItem** — confere: produto existe e está ativo? Se o produto exige sabor, veio sabor? O sabor é aceito por esse produto e está ativo? Então chama `cart.addItem( ... )` usando o **preço atual** do produto e salva.
- **UpdateCartItem** — muda a quantidade de uma linha.
- **RemoveCartItem** — tira uma linha.
- **ClearCart** — esvazia.
- `cart-view.ts` — um ajudante que transforma o carrinho em um "resumo para tela": para cada item, busca o nome do produto e do sabor e monta um JSON bonitinho com subtotais e total.

### Pedidos (precisa estar logado)

- **CreateOrder** ( `POST /orders` ) — pega o carrinho; recusa se estiver vazio. Para cada item, relê o produto/sabor (ainda existe? ainda ativo?), monta a lista "congelada" com nome + preço atuais, cria o `Order` , salva, e por fim **esvazia o carrinho**.
- **ListOrders** ( `GET /orders` ) — pedidos do cliente logado, paginados.
- **GetOrder** ( `GET /orders/:id` ) — um pedido. Só o *dono* ou um *admin* pode ver; senão, `ForbiddenError` .

## 12 Providers (as ferramentas reais)

`src/infra/providers/` — aqui os contratos (ports) ganham corpo. Todas usam só bibliotecas que **já vêm no Node.js** ( `node:crypto` ), sem instalar nada extra.

| Arquivo / classe                                                 | Cumpre o port                                            | Como                                                                                                                                                                  |
|------------------------------------------------------------------|----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>CryptoHashProvider</code>                                  | <code>HashProvider</code>                                | <b>script</b> com "sal" aleatório. Guarda <code>script\$sal\$hash</code> . Compara em tempo constante (evita ataque de tempo).                                        |
| <code>HmacTokenProvider</code>                                   | <code>TokenProvider</code>                               | Monta um token no <b>formato JWT</b> (cabeçalho.dados.assinatura) e assina com HMAC-SHA256 usando o <code>JWT_SECRET</code> . Sem biblioteca de JWT.                  |
| <code>ConsoleMailProvider</code>                                 | <code>MailProvider</code>                                | Não envia e-mail de verdade: <b>imprime a mensagem no terminal</b> (inclusive o código). Ótimo para estudar. Em produção, troca por um que fale com um servidor SMTP. |
| <code>UuidGenerator</code> /<br><code>CryptoCodeGenerator</code> | <code>IdGenerator</code> /<br><code>CodeGenerator</code> | <code>randomUUID()</code> para ids; <code>randomInt(0,10)</code> repetido para o código de 6 dígitos.                                                                 |

💡 **Dica** — Repare no padrão: **o nome do arquivo diz a tecnologia** ( `crypto-` , `hmac-` , `console-` ). Se um dia entrar `bcrypt-hash-provider.ts` ou `nodemailer-mail-provider.ts` , basta trocar uma linha no container. O resto do sistema nem percebe.

## 13 Repositórios com Prisma

`src/infra/database/prisma/` : `client.ts` (cria o objeto `prisma` ) e um repositório por assunto. Cada um **implementa** a interface do domínio e faz o "**de-para**" (mapper) entre a linha do banco e a entidade.

```
function toDomain(row) {           // linha do banco → entidade
  return Customer.restore({
    id: row.id,
    email: Email.unsafe(row.email),
    role: row.role === 'ADMIN' ? 'ADMIN' : 'CUSTOMER',
    // ...
  })
}

async findByEmail(email) {
  const row = await this.prisma.customer.findUnique({ where: { email } })
  return row ? toDomain(row) : null
}
```

Detalhes que valem citar:

- **Produto ↔ sabores:** ao salvar, apaga as ligações antigas em `ProductFlavor` e recria, dentro de uma **transação** (tudo ou nada).
- **Carrinho:** `save()` faz "upsert" do carrinho e **substitui todas as linhas** de uma vez, também em transação. Simples e sempre consistente.
- **Pedido:** `create()` grava o pedido e todos os itens juntos.

🗨 **Analogia** — O mapper é um **tradutor juramentado** na fronteira: de um lado fala "linguagem de banco" (colunas, texto), do outro "linguagem de negócio" (entidades, `Money`, `Email`). Nenhum dos dois lados precisa aprender o idioma do outro.

## 14 A camada HTTP (Fastify)

`src/infra/http/` . É a "recepção" que recebe pedidos da internet e chama o caso de uso certo.

- `http-helpers.ts` :
  - `parseWith(schema, dados)` — valida o corpo/query com **Zod** (biblioteca de validação). Se falhar, vira `ValidationError` com a lista de problemas.
  - `sendResult(reply, either, status)` — se `ok`, responde com o valor e o status de sucesso; se erro, responde `erro.status + { error: code, message }`.
- `auth.ts` — fábrica de "guardas" (middlewares que rodam *antes* da rota):
  - `authenticate` — lê o cabeçalho `Authorization: Bearer ...`, valida o token, e pendura `request.user = { id, role }`. Sem token válido → 401.
  - `authorizeAdmin` — depois do `authenticate`, exige `role === "ADMIN"`; senão → 403.
- `routes/*.ts` — um arquivo por assunto ( `customer`, `auth`, `product`, `flavor`, `cart`, `order` ). Cada rota: valida a entrada → chama `container.useCases.xxx.execute( ... )` → `sendResult( ... )`. As rotas são "burras" de propósito: **nenhuma regra de negócio aqui**.
- `app.ts` — monta o Fastify, registra cada grupo de rotas em um **escopo isolado** (para o guarda do carrinho não "vazar" para as rotas públicas), e define o **tratador de erros global** (traduz `AppError` e erros do Zod) e o "rota não encontrada" (404).

## 15 O container (composition root)

`src/container.ts` é o **único lugar** que sabe quais peças concretas existem. Ele:

1. cria os *providers* (hash, token, mail, geradores);
2. cria os *repositórios* Prisma, passando o `prisma`;
3. cria cada *caso de uso*, injetando os repositórios e ports de que ele precisa;
4. devolve um objeto `{ tokens, useCases: { ... } }` que as rotas usam.

❓ **Por que fizemos assim** — Isto é **injeção de dependência** feita "na mão". Os casos de uso nunca fazem `new PrismaCustomerRepository()` — eles *recebem* um `CustomerRepository` pronto. Num teste, você injeta um repositório falso em memória e testa a regra sem banco nenhum.

## 16 Servidor, variáveis de ambiente e seed

- `src/config/env.ts` — lê o arquivo `.env` e entrega valores já tratados: `port`, `jwtSecret`, `tokenTtlSeconds`, `passwordResetTtlMinutes`. Avisa no console se o segredo do token ainda é o de exemplo.
- `src/server.ts` — o "liga": `buildApp()` e `app.listen()` na porta 3389.
- `prisma/seed.ts` — cria o **administrador inicial** (`admin@julia.dev` / `admin12345`), porque sem um admin ninguém consegue cadastrar produto. Rodável com `npm run db:seed`.
- `package.json` — atalhos: `dev`, `start`, `typecheck`, `db:migrate`, `db:seed`, `db:studio`. Única dependência nova: `zod`.

## 17 Ajuste do TypeScript

O `tsconfig.json` estava num modo que **exige escrever `.js` no fim de cada import** — mas o código (inclusive o que já existia) não fazia isso. Trocamos para o modo `"moduleResolution": "Bundler"`, que combina com a forma como o projeto roda (via `tsx`). Resultado: `npm run typecheck` passa limpo, sem precisar reescrever centenas de imports.

## 18 Testes

**1) Verificação de tipos:** `npx tsc --noEmit` → zero erros.

**2) Teste de ponta a ponta** com o servidor no ar e `curl`, exercitando o fluxo real:

- cadastrar cliente → `201` com os dados (sem senha);
- login do cliente e do admin → token devolvido;
- admin cria sabor e produto → `201`;
- cliente tentando criar sabor → `403` (guarda de admin funcionando);
- adicionar ao carrinho, ver carrinho, criar pedido → total correto (R\$ 50,00 = 2 × 2500);
- esqueci a senha → código impresso no terminal; reset com código errado → `401`; com o certo → senha trocada; login com a senha antiga falha, com a nova funciona;
- editar produto e sabor ( `PUT` ), mudar quantidade ( `PATCH` ), remover item, criar pedido com carrinho vazio → `400`; pedir sem token → `401`.

Todos passaram.

## Parte 4 — Fluxos completos narrados

---

### Fluxo A: cadastro e primeiro login

1. App envia `POST /customers` com nome, e-mail e senha.
2. A rota valida o formato com `Zod` e chama `RegisterCustomer`.
3. O caso de uso cria `Email` e `RawPassword` (objetos de valor). Se algum for inválido → `400`.
4. Pergunta ao `CustomerRepository` se o e-mail já existe → se sim, `409`.
5. Pede ao `HashProvider` o hash da senha.
6. Monta `Customer.create( ... )` e chama `repo.create( ... )` → Prisma grava a linha.
7. Responde `201` com `customer.toPublic()` (sem o hash).
8. App então faz `POST /sessions`; `AuthenticateCustomer` confere a senha e devolve um **token**. O app guarda esse token.

### Fluxo B: recuperação de senha

1. `POST /password/forgot` com o e-mail. Resposta é sempre `202` ("se existir, enviamos") — não revela se a conta existe.
2. Existindo a conta: códigos antigos são invalidados; um novo código de 6 dígitos é gerado; guarda-se o **hash** dele com validade de 15 min; o `MailProvider` "envia" (aqui, imprime no terminal).
3. `POST /password/reset` com e-mail + código + nova senha.
4. `ResetPassword` valida a nova senha, busca o último código utilizável, compara o hash. Errado/expirado → `401`.
5. Certo: grava o novo hash de senha, marca o código como **consumido** (uso único) e invalida os demais.

### Fluxo C: admin cadastra produto

1. `POST /admin/flavors` (com token de admin) cria "Chocolate". Sem token → `401`. Token de cliente comum → `403`.
2. `POST /admin/products` com nome, `priceCents` e `flavorIds: ["id-do-chocolate"]`.
3. `CreateProduct` valida o preço com `Money.fromCents` e confere que *todo* id de sabor existe.
4. Grava o produto e as ligações `ProductFlavor` numa transação.

### Fluxo D: cliente compra

1. `POST /cart/items` com `productId`, `flavorId` e `quantity: 2`.
2. `AddCartItem` confere produto ativo, sabor válido e ativo, pega o **preço atual** e chama `cart.addItem( ... )`. O carrinho é salvo.
3. `GET /cart` mostra o resumo: item, subtotal 5000, total 5000.

4. `POST /orders` . `CreateOrder` relê os produtos, "congela" nome+preço, cria o `Order` (status `PENDING` , total 5000) e **esvazia o carrinho**.
5. `GET /orders` lista o pedido; `GET /orders/:id` mostra o detalhe (só para o dono ou admin).

## Parte 5 — Mapa de endpoints

| Recurso                        | Método e rota                     | Quem pode      |
|--------------------------------|-----------------------------------|----------------|
| Cadastro de clientes           | POST /customers                   | qualquer um    |
| Login                          | POST /sessions                    | qualquer um    |
| Pedir código de recuperação    | POST /password/forgot             | qualquer um    |
| Redefinir senha com o código   | POST /password/reset              | qualquer um    |
| Catálogo (só ativos)           | GET /products , GET /products/:id | qualquer um    |
| Listar produtos incl. inativos | GET /admin/products               | admin          |
| Cadastrar produto              | POST /admin/products              | admin          |
| Editar produto                 | PUT /admin/products/:id           | admin          |
| Listar sabores                 | GET /flavors                      | qualquer um    |
| Cadastrar sabor                | POST /admin/flavors               | admin          |
| Editar sabor                   | PUT /admin/flavors/:id            | admin          |
| Ver carrinho                   | GET /cart                         | cliente logado |
| Adicionar item                 | POST /cart/items                  | cliente logado |
| Mudar quantidade               | PATCH /cart/items/:itemId         | cliente logado |
| Remover item                   | DELETE /cart/items/:itemId        | cliente logado |
| Esvaziar carrinho              | DELETE /cart                      | cliente logado |
| Criar pedido                   | POST /orders                      | cliente logado |
| Meus pedidos                   | GET /orders                       | cliente logado |
| Detalhe do pedido              | GET /orders/:id                   | dono ou admin  |
| Saúde do serviço               | GET /health                       | qualquer um    |

## Parte 6 — Como rodar

---

```
npm install
npm run db:migrate      # cria as tabelas no dev.db
npm run db:seed         # cria o admin: admin@julia.dev / admin12345
npm run dev             # sobe a API em http://localhost:3389

# verificar tipos:
npm run typecheck

# olhar o banco numa tela:
npm run db:studio
```

**Variáveis em `.env`** : `DATABASE_URL` , `PORT` , `JWT_SECRET` (troque em produção!), `TOKEN_TTL_SECONDS` , `PASSWORD_RESET_TTL_MINUTES` , `ADMIN_EMAIL` , `ADMIN_PASSWORD` .

⚠ **Atenção** — O código de recuperação de senha aparece **no terminal onde o servidor está rodando** (é o `ConsoleMailProvider` ). Em produção, implemente um `MailProvider` de verdade.

## Parte 7 — Glossário

| Termo                  | Em palavras simples                                                                                |
|------------------------|----------------------------------------------------------------------------------------------------|
| API                    | Lista de "pedidos" que o backend aceita.                                                           |
| Endpoint / rota        | Um endereço da API (ex.: <code>/orders</code> ).                                                   |
| Entidade               | Coisa do negócio com identidade própria e regras (Cliente, Pedido).                                |
| Objeto de valor        | Coisa definida pelo conteúdo, sem id, imutável (E-mail, Dinheiro).                                 |
| Agregado               | Um grupo de objetos tratado como unidade, com uma "raiz" que controla o acesso (Carrinho + itens). |
| Caso de uso            | O passo a passo de uma tarefa do sistema ("Criar pedido").                                         |
| Repositório            | Objeto que guarda/busca entidades. No domínio é só a interface; a implementação usa Prisma.        |
| Port / Adapter         | Port = contrato (interface). Adapter = a peça real que cumpre o contrato.                          |
| DTO                    | "Data Transfer Object": um objeto simples só para levar dados de entrada/saída.                    |
| ORM                    | Tradutor entre objetos do código e tabelas do banco (Prisma).                                      |
| Migration              | Arquivo que registra uma mudança na estrutura do banco.                                            |
| Hash                   | Embaralhamento só de ida: dá para conferir, não dá para "desembaralhar".                           |
| Token (JWT)            | Texto assinado que prova quem você é depois do login.                                              |
| Middleware / guard     | Função que roda antes da rota (ex.: checar login).                                                 |
| Injeção de dependência | Entregar as peças prontas para uma classe em vez de ela mesma criá-las.                            |
| Composition root       | O único lugar que monta o quebra-cabeça de peças concretas ( <code>container.ts</code> ).          |
| Seed                   | Script que insere dados iniciais (o admin).                                                        |
| Snapshot               | "Foto" de um dado num momento (nome/preço no <code>OrderItem</code> ).                             |

## Parte 8 — Exercícios para fixar

---

1. Desenhe as camadas de memória e diga, para cada arquivo abaixo, em qual camada ele está e por quê: `customer.ts` , `register-customer.ts` , `prisma-customer.repository.ts` , `customer-routes.ts` .
2. Por que `RegisterCustomer` não faz `new CryptoHashProvider()` lá dentro? O que isso facilita nos testes?
3. Explique por que o preço fica em centavos. Dê um exemplo numérico de erro que isso evita.
4. No login, por que a mensagem de erro é a mesma para "e-mail não existe" e "senha errada"?
5. Em `/password/forgot` , por que respondemos "ok" mesmo quando o e-mail não existe?
6. O que aconteceria se `OrderItem` guardasse só `productId` e o produto fosse apagado depois? Como o snapshot resolve isso?
7. Siga com o dedo o caminho de um `POST /orders` : rota → caso de uso → entidades → repositórios → resposta. Cite os arquivos.
8. Quer trocar o e-mail "de terminal" por um real. Quais arquivos você cria/edita? (Resposta: um novo adapter que implementa `MailProvider` + uma linha no `container.ts` .)
9. Adicione mentalmente um caso de uso "Cancelar pedido". Que método da entidade `Order` ele chamaria? Que erro retornaria se o pedido já estivesse pago?
10. Por que as rotas do carrinho ficam num "escopo isolado" no `app.ts` ?

---

Gerado como material de estudo do projeto **backend-julia**. Consulte também o `README.md` na raiz do projeto e explore o código na ordem das etapas acima — comece por `src/domain/` , depois `src/application/` , depois `src/infra/` .